David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Modern Large Language Models

① LLMs types
② Model pretraining
③ Alignment methods
④ Optimization and scaling
⑤ Open vs. closed LLMs
⑥ Summary and Takeaways

Objectives

- Differentiate major LLM types: **AR, AE, Seq2Seq, RAG, Multimodal**.
- Understand **pretraining objectives** (CLM, MLM, denoising, contrastive).
- Explain **alignment methods**: Instruction Tuning, RLHF, DPO.
- Analyze key **optimization and scaling techniques**.
- Compare **open vs. closed commercial LLM ecosystems**.

LLMs Types

Main Types of Language Models

- **Autoregressive Language Models (AR)**
  - Decoder-only architecture. Trained to predict the next token (e.g., GPT, LLaMA, Falcon)
- **Autoencoding Language Models (AE)**
  - Encoder-only architecture. Trained to reconstruct masked tokens (e.g., BERT, RoBERTa)
- **Sequence-to-Sequence Language Models (Seq2Seq)**
  - Encoder-decoder architecture. Trained for text-to-text tasks (e.g., T5, BART, UL2)
- **Retrieval-Augmented Language Models (RAG)**
  - Combine a retriever with a generator. Use external documents for generation (e.g., RAG, Atlas, Command R+)
- **Multimodal Language Models**
  - Process and generate multiple modalities (text, image, audio; GPT-4, Gemini, Flamingo, CLIP)
- **Instruction-Following / Aligned Language Models**
  - Fine-tuned with supervised instructions and human preferences (ChatGPT, Claude, Alpaca, OpenChat)

Autoregressive Language Models (AR / Causal)

- Predict the next token given all previous tokens in a left-to-right (causal) manner.
- **Architecture**: Decoder-only Transformer
- **Training Objective**: Causal Language Modeling (CLM)
- **Examples**:
  - **GPT** (GPT-2, GPT-3, GPT-4)
  - **LLaMA**, **Mistral**, **Falcon**, **Gemma**
  - **CodeGen**, **Yi**
- **Use Cases**:
  - Text generation, dialogue agents, code generation

Autoencoding Language Models (AE / Masked)

- Learn bidirectional representations by reconstructing masked tokens within input sequences.
- **Architecture**: Encoder-only Transformer
- **Training Objective**: Masked Language Modeling (MLM)
- **Examples**:
  - **BERT**, **RoBERTa**, **DeBERTa**, **ERNIE**
- **Use Cases**:
  - Sentence classification, QA, token classification (NER, POS tagging)

Seq2Seq / Encoder-Decoder

- Encode the input sequence and generate a corresponding output sequence (text-to-text).
- **Architecture**: Encoder-Decoder Transformer
- **Training Objective**: Denoising, translation, summarization tasks
- **Examples**:
  - **T5**, **BART**, **UL2**, **mT5**
  - **Flan-T5**, **Pegasus**, **MBART**
- **Use Cases**:
  - Translation, summarization, question answering, instruction following

Retrieval-Augmented Language Models (RAG)

- Combine parametric knowledge (in the model) with non-parametric knowledge (external retriever).
- **Architecture**: Encoder + Retriever + Generator
- **Training Objective**: RAG loss, contrastive pretraining, fine-tuning on task-specific data
- **Examples**:
  - **RAG**, **REALM**, **RETRO**, **Atlas**, **Command R+**
- **Use Cases**:
  - Open-domain QA, knowledge-intensive generation, enterprise search

Multimodal Language Models

- Process and generate across multiple modalities (text, image, audio, video).
- **Architecture**: Transformer + vision encoder / cross-modal attention
- **Training Objective**: Contrastive learning, captioning loss, autoregressive generation
- **Examples**:
  - **GPT-4 (multimodal)**, **Gemini**, **Flamingo**, **CLIP**, **Grok V**
  - **MME**, **Kosmos**, **BLIP-2**, **GIT**
- **Use Cases**:
  - Visual QA, captioning, image/text retrieval, video QA

Instruction-Following / Aligned LMs

- Finetuned on instructions, preferences, and dialogues to follow user intent.
- **Architecture**: Usually decoder-only (GPT-style)
- **Training Objective**: Instruction tuning, RLHF, DPO
- **Examples**:
  - **InstructGPT**, **ChatGPT**, **Claude**, **OpenChat**, **Alpaca**, **Zephyr**
- **Use Cases**:
  - Conversational AI, copilots, assistants, tutoring

Summary

Type	Architecture	Objective	Example Models	Use Cases
Autoregressive (AR)	Decoder-only	Causal LM	GPT, LLaMA, Falcon	Generation, chatbots, code
Autoencoding (AE)	Encoder-only	Masked LM	BERT, RoBERTa, DeBERTa	Classification, QA, NER
Sequence-to-Sequence (Seq2Seq)	Encoder-Decoder	Denoising / Translation	T5, BART, mT5	Summarization, translation, QA
Retrieval-Augmented (RAG)	Hybrid	Retrieval + generation	RAG, Atlas, Command R+	Knowledge-based generation
Multimodal LMs	Cross-modal	CLM + vision tasks	GPT-4, Gemini, Flamingo, CLIP	VQA, image-text, captioning
Instruction-Following	Decoder-only	Supervised + preference	ChatGPT, Claude, Alpaca, OpenChat	Chat, assistance, alignment

GPTs and Applications

Generative Pretrained Transformers (GPTs)

- **Autoregressive large language models** built on the **Transformer decoder architecture**.
- Designed to **generate coherent text** by predicting the next token in a sequence.
- Pretrained on large unlabeled corpora using **causal language modeling**.
- **Key Characteristics of GPTs**
  - **Generative**: Trained to produce text, token by token.
  - **Pretrained**: Trained on diverse text before any task-specific tuning.
  - **Transformer-based**: Use self-attention and FFN to model long-range dependencies.
- **Architecture Overview**
  - **Input**: Tokenized sequence with positional encodings.
  - **Stacked Decoder Blocks**:
    - Masked Multi-head Self-Attention
    - Feed-Forward Network (FFN)
    - Layer Norm + Residual Connections
  - **Output**: Distribution over vocabulary for the next token.

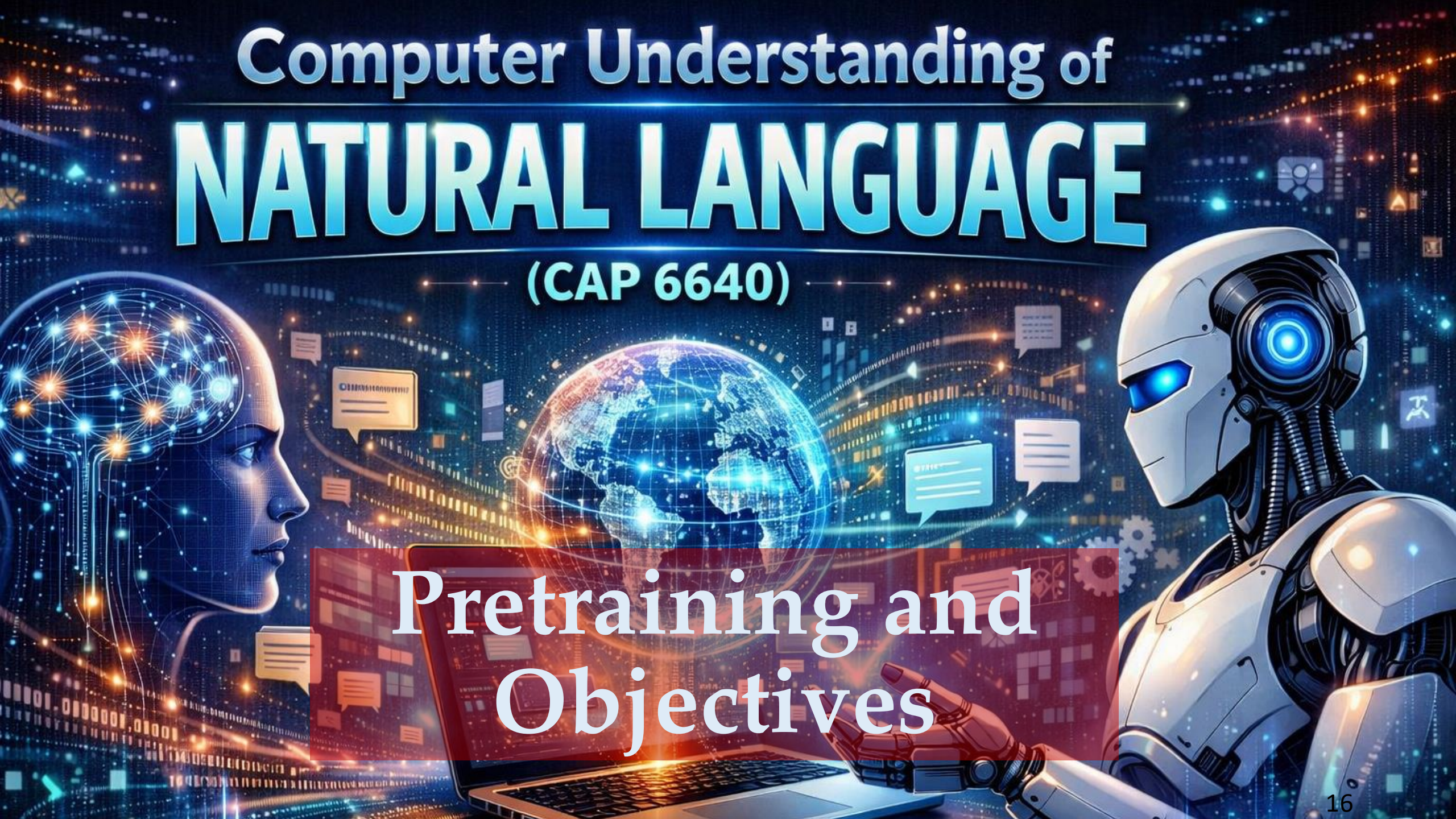
Applications of GPTs

- Chatbots and Conversational Agents
- Text Summarization
- Machine Translation
- Question Answering
- Code Generation (e.g., GitHub Copilot)
- Creative Writing and Story Generation
- Instruction Following and Tutoring
- …

Model	Release Year	Parameters	Notable Features
GPT-1	2018	117M	Proof-of-concept for unsupervised LM
GPT-2	2019	1.5B	Coherent text generation, no release at first
GPT-3	2020	175B	Few-shot learning, general-purpose NLP
GPT-4	2023	Undisclosed	Multimodal capabilities, RLHF alignment

Deep Dive into Building LLMs/GPTs

- Stages of Pretraining
  - CLM, MLM, Denoising, Contrastive Learning
- Alignment and Fine-tuning Methods
  - Instruction Tuning, Reinforcement Learning, DPO
- Optimization and Scaling
  - Parallelism, Mixed Precision, Data Curation, and Curriculum Learning



Pretraining and Objectives

Stages of LLM Training

- **Pretraining**
  - Learns general-purpose language representations.
  - Trained on massive unlabeled corpora using self-supervised objectives.
- **Instruction Tuning**
  - Fine-tunes the model on datasets where inputs are paired with desired outputs.
  - Helps the model better follow human instructions.
- **Reinforcement Learning from Human Feedback (RLHF)**
  - Aligns the model with human preferences.
  - Involves training a reward model and optimizing via reinforcement learning.

Pretraining Methods

- **Causal Language Modeling (CLM)**
  - Predicts the next token given previous tokens.
  - Used in models like GPT, LLaMA.
- **Masked Language Modeling (MLM)**
  - Randomly masks tokens and asks the model to predict them.
  - Used in BERT, RoBERTa.
- **Denoising Objectives**
  - Corrupts input and asks the model to reconstruct it.
  - Used in T5, BART.
- **Contrastive Learning**
  - Trains the model to distinguish correct completions from incorrect ones.
  - Used in some multimodal or retrieval-augmented settings.

Causal Language Modeling (CLM)

- **Definition**: CLM is a *unidirectional* language modeling objective where the model predicts the next token xt given all previous tokens x1,x2,...,xt−1.
- **Formulation**:

$$\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t}; \theta)$$

where θ are model parameters, and the context is strictly left-to-right.

- **Architecture**: Uses **causal (masked) self-attention**, ensuring each position attends only to past positions.
- **Usage**:
  - Core objective in **GPT**, **LLaMA**, **Mistral**, **Falcon**, **Yi**, etc.
  - Enables fluent generation and reasoning with long-range dependencies.
  - Typically trained on diverse, large-scale text (e.g., Common Crawl, books, code).
- **Pros**: Suited for autoregressive generation tasks.
- **Cons**: Can't naturally leverage bidirectional context during training.

Masked Language Modeling (MLM)

Masked Language Modeling (MLM)

- **Definition**: MLM is a *bidirectional* objective where the model predicts missing (masked) tokens within a sequence.
- **Formulation**:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P(x_i \mid x_{\setminus \mathcal{M}}; \theta)$$

where $\mathcal{M}$ is the set of masked positions.

- **Masking Strategy**:
  - 80% of the time: Replace with [MASK]
  - 10%: Replace with a random token
  - 10%: Keep unchanged (but still predict)
- **Architecture**: Uses **full self-attention** (bidirectional), enabling each token to attend to all others.
- **Usage**:
  - Foundation of **BERT**, **RoBERTa**, **DeBERTa**, **ERNIE**.
  - Suitable for representation learning in NLP tasks (e.g., classification, QA).
- **Pros**: Learns rich contextual embeddings from both directions.
- **Cons**: Inefficient for generative tasks; pretraining-task mismatch for generation.

Denoising Objectives

- General class of objectives where the model reconstructs a clean output from a corrupted (noisy) input.
- **Formulation (T5-style span corruption)**:
  - Randomly replace contiguous spans of tokens with sentinel tokens.
  - Task: Predict the missing spans in order.
  - Loss applied only to target (missing) tokens:

$$\mathcal{L}_{\text{T5}} = -\sum_{\text{spans}} \log P(\text{span}_j \mid \text{corrupted input}; \theta)$$

- **Examples**:
  - **BART**: Applies multiple noise types (token deletion, permutation, span masking).
  - **T5**: Uses span masking with sentinel tokens in a text-to-text format.
- **Architecture**:
  - **BART**: Full encoder-decoder Transformer.
  - **T5**: Unified text-to-text encoder-decoder.
- **Usage**: **T5**, **BART**, **UL2**, and other seq2seq models for text generation, summarization, translation.
- **Pros**: Flexible and expressive; fits text-to-text settings.
- **Cons**: More complex to implement and tune; requires careful masking schemes.

Contrastive Learning

- Contrastive learning aligns semantically similar inputs by pulling together positive pairs and pushing apart negative pairs in representation space.
- **Formulation** (InfoNCE loss):

$$\mathcal{L}_{\mathrm{CL}} = -\log \frac{\exp(\mathrm{sim}(q, k^+)/\tau)}{\sum_{k \in \mathcal{K}} \exp(\mathrm{sim}(q, k)/\tau)}$$

  - q: query, k^+: positive key, K: set of keys (incl. negatives), τ: temperature, sim(·) : similarity (e.g., cosine).
- **Variants**:
  - **SimCSE** (sentence embeddings): Contrastive fine-tuning on NLI datasets.
  - **CLIP**: Image-text pairs.
  - **RAG/RETRO**: Uses contrastive loss to match queries with relevant retrievals.
- **Usage**:
  - Multimodal LLMs (e.g., CLIP, Flamingo)
  - Retrieval-augmented models (e.g., RAG, Atlas)
  - Embedding-based search and clustering
- **Pros**: Improves embedding quality and discrimination.
- **Cons**: Requires carefully sampled negatives; sensitive to batch size and temperature.

Pretraining Methods, Comparison

Aspect	Causal Language Modeling (CLM)	Masked Language Modeling (MLM)	Denoising Objectives	Contrastive Learning
Objective	Predict next token given previous tokens	Predict randomly masked tokens	Reconstruct original text from corrupted input	Differentiate positive from negative examples
Directionality	Unidirectional (left-to-right)	Bidirectional	Bidirectional (encoder-decoder)	Often bidirectional (depends on architecture)
Masking Strategy	None (causal masking in attention)	Random token masking (typically 15%)	Span masking, permutation, deletion, etc.	No masking; uses query-key pairs
Model Architecture	Decoder-only transformer	Encoder-only transformer	Encoder-decoder (T5, BART)	Encoder or dual-encoder (e.g., CLIP, SimCSE)
Main Use Cases	Autoregressive text generation, chatbots	Representation learning, classification	Text-to-text tasks (e.g., summarization, translation)	Embedding learning, retrieval, multimodal LMs
Example Models	GPT, LLaMA, Mistral, Falcon	BERT, RoBERTa, DeBERTa	T5, BART, UL2	CLIP, SimCSE, RAG, Flamingo
Strengths	Fluent generation, long-context modeling	Rich contextual understanding	Versatile, supports various NLP tasks	Strong semantic rep., retrieval-ready
Limitations	No bidirectional context	Not suitable for generative tasks	Computationally intensive, complex corruption setup	Sensitive to negative sampling and hyperparams

Alignment and Fine-tuning

Alignment and Fine-Tuning Methods

- **Instruction Tuning**
  - Uses curated prompt-response datasets.
  - Makes the model more task-oriented and helpful.
- **RLHF (Reinforcement Learning from Human Feedback)**
  - Generates multiple completions for a prompt.
  - Human annotators rank them.
  - A reward model is trained based on these rankings.
  - The base model is fine-tuned using reinforcement learning (e.g., PPO).
- **Direct Preference Optimization (DPO)**
  - A newer alternative to RLHF.
  - Uses preference data directly in the loss function, skipping PPO.

Instruction Tuning

- Supervised fine-tuning technique where the model learns to follow human instructions by training on curated input-output pairs.
- **Data Format**: Each sample is a prompt (instruction or task description) paired with an ideal response.
  - Input: "Translate ’hello’ to French"→Output: "bonjour”
- **Objective Function**: Cross-entropy loss over output tokens:

$$\mathcal{L}_{\text{SFT}} = -\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, x; \theta)$$

- **Training Setup**:
  - Often uses open instruction datasets (e.g., FLAN, Alpaca, OpenAssistant, ShareGPT).
  - No preference signal; trains directly on supervised outputs.
- **Strengths**: Simple, stable, and effective. Makes models useful for zero-shot and few-shot inference.
- **Limitations**: Alignment limited to training examples. Can't correct misaligned generation behavior beyond dataset coverage.

Reinforcement Learning from Human Feedback

- Aligns LLM outputs with human preferences by introducing a reward signal derived from ranked completions.
- **Pipeline**:
  - **Supervised Fine-Tuning (SFT)**: Initialize from instruction tuning.
  - **Reward Model (RM)**: Train a model R(x,y) to predict human preferences from ranked completions.
  - **Policy Optimization**: Use RL (typically PPO) to optimize the model to generate outputs with higher reward values.
- **Reward Model Loss**:

$$\mathcal{L}_{\text{RM}} = -\log \frac{\exp(R(x, y^+)/\beta)}{\exp(R(x, y^+)/\beta) + \exp(R(x, y^-)/\beta)}$$

- **RL Objective (PPO)**:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_{\pi}\left[\frac{\pi(y \mid x)}{\pi_{\text{old}}(y \mid x)} A(y, x) - \text{KL}[\pi \| \pi_{\text{old}}]\right]$$

- **Strengths**: Can fine-tune behavior without explicit gold responses. Models like GPT-4 and Claude use it to align to human values.
- **Limitations**: Expensive (requires human annotators). PPO is unstable and complex to train.

Direct Preference Optimization (DPO)

- A direct supervised learning alternative to RLHF that avoids training a reward model or using RL. It directly uses preference pairs to adjust the policy.
- **Objective Function**: For a pair (y^+, y^-), DPO minimizes:

$$\mathcal{L}_{\text{DPO}} = -\log \frac{\exp(\beta \log \pi(y^+ \mid x))}{\exp(\beta \log \pi(y^+ \mid x)) + \exp(\beta \log \pi(y^- \mid x))}$$

  where β controls sharpness of preference.
- **Key Properties**:
  - Theoretically derived from RL reward maximization under KL constraint.
  - No reward model or sampling needed; directly optimizes preferences.
- **Strengths**:
  - More stable and efficient than PPO.
  - Better sample efficiency; easier to implement.
- **Limitations**:
  - Still requires quality preference data.
  - Less explored in multimodal or long-context alignment.

Instruction Tuning, Comparison

Aspect	Instruction Tuning	RLHF (PPO)	Direct Preference Optimization (DPO)
Training Signal	Supervised (gold outputs)	Human-ranked preferences	Human preference pairs
Optimization Type	Cross-entropy loss	Reinforcement Learning (e.g., PPO)	Direct log-likelihood preference loss
Need for Reward Model	No	Yes	No
Stability	High	Low (PPO sensitive to hyperparameters)	High
Sample Efficiency	High	Low (requires many rollouts and comparisons)	High
Implementation Complexity	Low	High	Moderate
Use Cases	Task-oriented generation	Human-aligned chat, safety fine-tuning	Efficient preference-based alignment
Example Models	FLAN-T5, Alpaca, Dolly	GPT-4, Claude, InstructGPT	OpenChat-DPO, TRLX-DPO fine-tuned models

Optimization and Scaling

Optimization and Scaling Techniques

- **Parallelism**
  - Data Parallelism: replicate the model across GPUs.
  - Model Parallelism: split the model across devices.
  - FSDP / ZeRO: optimize memory usage in training.
- **Mixed Precision Training**
  - Uses FP16 or bfloat16 to reduce memory and increase speed.
- **Dataset Curation**
  - Involves filtering, deduplication, and cleaning to ensure high-quality training data.
- **Curriculum Learning**
  - Trains the model progressively on increasingly complex tasks or data.

Parallelism

- Techniques that split model computation or data across multiple devices to enable training of large models.
- **Types**:
  - **Data Parallelism**: Each device has a copy of the model; processes different input batches. Gradients are averaged across devices.
  - **Model Parallelism**: Model parameters are split across devices (e.g., tensor parallelism, pipeline parallelism).
  - **FSDP (Fully Sharded Data Parallel)** / **ZeRO**: Advanced strategies to shard optimizer states, gradients, and activations across devices to reduce memory overhead.
- **Used in**: GPT-3, LLaMA, Mixtral, PaLM

Mixed Precision Training

- Use of lower-precision number formats (e.g., FP16 or bfloat16) for faster computation and reduced memory usage without compromising model accuracy.
- **Benefits**:
  - Reduces GPU memory footprint (up to 50%)
  - Improves throughput on hardware like NVIDIA Tensor Cores
  - Often combined with loss scaling to maintain numerical stability
- **Used in**: All modern LLM training pipelines (e.g., GPT, T5, Falcon)

Dataset Curation

- Preprocessing and filtering of training data to improve model quality and prevent learning from noisy or harmful content.
- **Techniques**:
  - Deduplication (exact/near-duplicate removal)
  - Language and domain filtering
  - Toxicity filtering and quality scoring
  - Controlling balance across domains (code, books, web)
- **Impact**: Improves generalization, fairness, safety, and downstream performance.
- **Used in**: OpenAI's GPT, Meta's LLaMA, Google's PaLM, T5

Curriculum Learning

- A training paradigm where the model is exposed to easier tasks or cleaner data first, followed by harder examples.
- **Types**:
  - Length-based (short → long sequences)
  - Complexity-based (simple → complex tasks)
  - Quality-based (cleaner → noisier samples)
- **Benefits**:
  - Faster convergence
  - Improved stability
  - Better generalization, especially in early training
- **Used in**: Gopher, Chinchilla, DeepMind's Gemini series

Optimization and Scaling, Comparison

Aspect	Parallelism	Mixed Precision Training	Dataset Curation	Curriculum Learning
Goal	Scale model training across hardware	Reduce memory and speed up training	Improve training data quality	Improve training efficiency and generalization
Techniques	Data parallelism, model parallelism, FSDP, ZeRO	FP16, bfloat16, loss scaling	Filtering, deduplication, scoring	Length-based, complexity-based, quality-based
Impact on Resources	Enables training larger models	Reduces memory usage and compute cost	Reduces noise, improves learning signals	Smooths training dynamics
Hardware Dependencies	Requires multi-GPU or multi-node setup	Supported on modern GPUs (e.g., A100)	No special requirements	No special requirements
Challenges	Communication overhead, implementation complexity	Numerical instability if not scaled	High curation cost, potential bias	Curriculum design is non-trivial
Commonly Used In	GPT-3, LLaMA, PaLM, Mixtral	Almost all LLMs	GPT, LLaMA, PaLM, T5	Gopher, Gemini, Chinchilla

Open vs. Closed LLMs

Types of Commercial LLMs

- Commercial LLMs vary by:
  - **Source availability**: Open vs. closed
  - **Deployment model**: API vs. self-hosted
  - **Optimization target**: Chat, code, search, RAG, multimodal

Type	Source	Access	Example Models
Open-Source	Open	Self-hostable	LLaMA, Mistral, Gemma
Closed-Source (API)	Proprietary	Hosted	GPT-4, Claude, Gemini
Open Weights (Restricted)	Semi-open	Limited terms	LLaMA 2/3 (Meta)
Fine-tuned Wrappers	Open+Tuned	Mixed	OpenChat, Alpaca

Open-Source Commercial LLMs

- **Key Characteristics**
  - Public weights and architecture
  - Permissive licenses (Apache 2.0, MIT)
  - Can be fine-tuned and deployed on-prem
- **Popular Models**
  - **LLaMA 3** (Meta)
  - **Mixtral** (Mistral AI)
  - **Gemma** (Google)
  - **Falcon** (TII)
  - **Yi / Qwen** (01.AI, Alibaba)

Open-Source Commercial LLMs

Model	Training Objective	Optimization Techniques	Datasets Used	# Parameters
LLaMA 2 / 3	Causal LM	FSDP, Mixed Precision, Custom Tokenizer	Common Crawl, GitHub, Wikipedia, Books, ArXiv	LLaMA 2: 7B / 13B / 70BLLaMA 3: TBD (expected 8B–400B)
Mistral / Mixtral	Causal LM	Sliding Window Attention, MoE (Mixtral), FP16	Refined web+code datasets (RedPajama subset)	Mistral: 7BMixtral: 12.9B (2/8 experts active)
Falcon	Causal LM	ZeRO, Data/Model Parallelism, Mixed Precision	Refined Common Crawl, curated corpora, books	Falcon-7B, Falcon-40B
Gemma	Causal LM	JAX/TPU Optimized, Mixed Precision	C4, Wikipedia, Books, code	Gemma-2B, Gemma-7B
Command R / R+	RAG-oriented Causal LM	Retrieval-Aware Finetuning, Mixed Precision	Instruction + RAG-augmented corpora	R+: 35B
Qwen	Causal LM (Chat Finetuned)	Tensor Parallelism, FP16, Custom Tokenizer	Multilingual corpus, Chinese & English corpora	Qwen-7B, Qwen-14B
Yi	Causal LM	FlashAttention, Deepspeed, Mixed Precision	EN/CH bilingual corpora, web, code, books	Yi-6B, Yi-34B
Dolly 2.0	Instruction Tuning (on Pythia)	Basic Finetuning, Mixed Precision	Databricks Instruction Dataset (15k prompt-response)	12B (Pythia base)
OpenChat	Instruction Tuning (LLaMA)	LoRA / DPO, QLoRA for efficiency	ShareGPT, OpenChat logs, instruction datasets	OpenChat-7B (LLaMA base)

Closed-Source Commercial LLMs

- **Key Characteristics**
  - Proprietary weights and training data
  - Accessible only via APIs
  - Continuous updates from providers

Leading Providers

- **OpenAI** – GPT-4, GPT-4 Turbo
- **Anthropic** – Claude 3 (Haiku, Sonnet, Opus)
- **Google DeepMind** – Gemini 1.5
- **Cohere** – Command R+

Semi-Open and Fine-Tuned Commercial Models

- **Fine-Tuned / Instruction-Tuned Models**
  - Derived from open base models
  - Targeted for chat, RAG, or domain-specific tasks
  - Often released by startups and communities
- **Examples**
  - **OpenChat** – Based on LLaMA
  - **Dolly 2.0** – From Databricks
  - **Vicuna / Orca / Zephyr** – Academic & open efforts

Model	Organization	License / Access / Status	Source Type	Notes
LLaMA 2 / 3	Meta	LLaMA License	Hybrid	Widely adopted; LLaMA 3 released in 2025 with limited-commercial terms.
Mistral / Mixtral	Mistral AI	Apache 2.0	Open-Source	Small but performant; Mixtral uses Mixture-of-Experts (MoE).
Falcon	Tech Innovation Inst	Apache 2.0	Open-Source	Strong performance; pretraining focused on scale.
Gemma	Google	Apache 2.0	Open-Source	Released in 2024; focused on responsible AI practices.
Command R / R+	Cohere for AI	Apache 2.0 (R), API (R+)	Hybrid	R is open, R+ is hosted; optimized for retrieval-augmented generation.
Qwen	Alibaba	Apache 2.0	Open-Source	Competitive multilingual performance.
Yi	01.AI	Apache 2.0	Open-Source	Top multilingual performer from Kai-Fu Lee's AI startup.
Dolly 2.0	Databricks	OpenRAIL	Open-Source	Based on open model (Pythia); instruction fine-tuned.
OpenChat	Community project	Various	Open-Source	Fine-tuned LLaMA; strong alignment and performance.
GPT-4 / GPT-4 Turbo	OpenAI	API (Azure/OpenAI)	Closed-Source	Industry-leading in reasoning and multi-modal tasks.
Claude 3	Anthropic	API	Closed-Source	Highly aligned; Claude 3 Opus leads benchmarks.
Gemini (1.5 series)	Google DeepMind	API (via Google Cloud)	Closed-Source	Multimodal with memory; PaLM successor.
Command R+ (Hosted)	Cohere	API	Closed-Source	Hosted RAG-optimized LLM.
Jurassic-2	AI21 Labs	API	Closed-Source	Strong alternative to GPT; good generation performance.
ERNIE Bot / SkyMind	Baidu	API (China)	Closed-Source	Chinese-market oriented model.
Tongyi Qianwen	Alibaba	API	Closed-Source	Cloud-integrated enterprise LLM.
Grok (xAI)	Elon Musk / xAI	API (via X)	Closed-Source	Embedded in X (Twitter); not open.
GPT-4 (leaked)	OpenAI	Not officially open	Closed-Source	No official release; some replication attempts.
Inflection AI (Pi)	Inflection AI	API	Closed-Source	Conversational, empathetic dialog model.

Takeaways

- **LLMs come in multiple forms**, including autoregressive, autoencoding, encoder-decoder, retrieval-augmented, and multimodal.
- **GPTs are autoregressive transformer models** trained to predict the next token using causal language modeling.
- **Instruction tuning, RLHF, and DPO are critical for aligning models** with human intent and producing helpful, safe outputs.
- **Pretraining methods** (CLM, MLM, denoising, contrastive) **define how models learn** linguistic and semantic patterns.
- Optimization techniques like parallelism, mixed precision, and curriculum learning are essential for scaling LLMs efficiently.
- LLMs support a wide range of applications, from chatbots and summarization to code generation and multimodal reasoning.